



TECHNICAL SPESIFICATION

CNN Trigger System

Abstract

The CNN Trigger System is a real-time, AI-based event selector designed to ingest high-speed raw data from front-end digitizers (ADC), perform low-latency inference using parallel CNN Accelerator Cores, and trigger the readout of high-fidelity waveform data.

The architecture employs a Split-Path Strategy to decouple data storage from analysis. Incoming data is bifurcated into a Storage Path (High-Fidelity Ground Truth) and an Inference Path (Downsampled/Selected Channels). A central PicoRV32 soft-core orchestrates a pool of parallel CNN Accelerators via a dedicated uDMA engine.



This template is intended for generic engineering documents only. Its use is subject to some recommendations. It must not be used for documents that are supporting specific processes such as Engineering Change Requests (ECR). Please see [all details](#) on the CERN ATS Quality Service website.

DOCUMENT PREPARED BY:

[Albert Chueng – UCI]

DOCUMENT TO BE CHECKED BY:

TBD

DOCUMENT TO BE APPROVED BY:

TBD

DOCUMENT SENT FOR INFORMATION TO:

OPTIONAL: I am a recipient of information

SUMMARY OF THE ACTIONS TO BE UNDERTAKEN:

OPTIONAL: I am a summary



Contents

1 Introduction	6
1.1 Top-Level Block Diagram	6
1.2 Key Subsystems	6
2 Functional Description	7
2.1 Split-Bank Memory Architecture	7
2.1.1 Ingress Signal Splitting (ingress_demux)	7
2.1.2 Banked Memory Roles	7
2.1.3 Addressing and Circularity	8
2.2 Task Orchestration	8
2.2.1 Dynamic "First-Available" Scheduler	8
2.3 Timing and Wrapper Logic	8
2.3.1 AI Accelerator Wrapper (cnn_core_wrapper)	8
2.3.2 Two-Tier Timing Engine	9
2.4 Trigger and Re-Ordering Logic	9
2.4.1 Software Re-Order Buffer	9
2.4.2 Thresholding and Backend Data Move	9
3 Memory Map and Register Description	10
3.1 Global Control Registers	10
3.2 Core-Specific Registers	10
4 Timing and Performance Constraints	10
5 Operational Modes	11
5.1 Idle Mode	11
5.2 Run Mode	11
5.3 Passthrough Mode	11
6 System Initialization and Error Recovery	11
6.1 Cold Boot Sequence	11
6.2 Run Start Procedure	11
A Appendix	12
A.1 Term and Parameter Definitions	12



List of Figures

1	Top Level Architecture of the CNN Trigger System	6
---	--	---



List of Tables

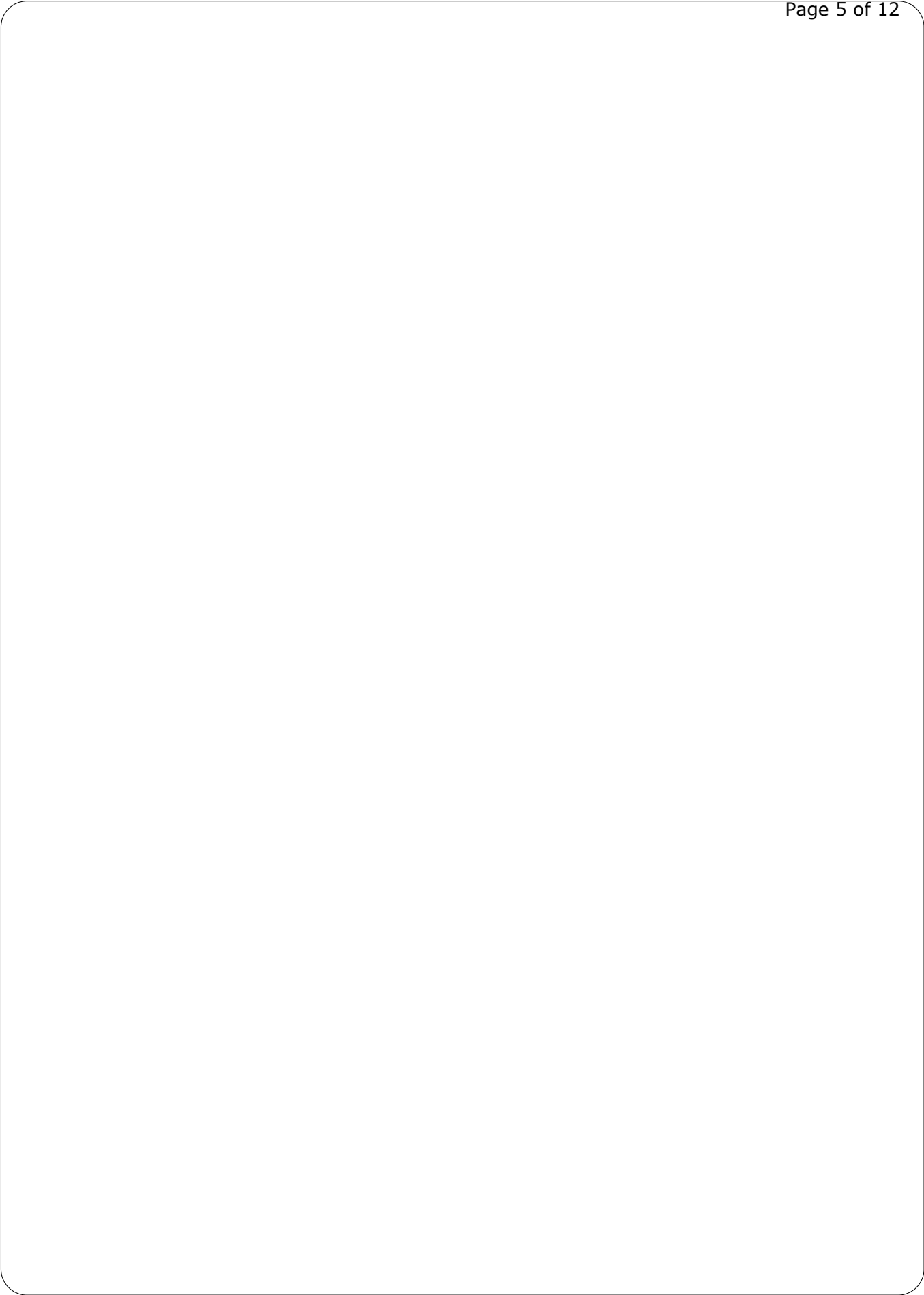
1	Global Control Registers Map	10
2	Core-Specific Registers (Offset from Core Base)	10
3	System Terms and Definitions	12



ARIANNA

REFERENCE
UCI-A-REF-0001

EDMS NO.	REV.	VALIDITY
-----	1.1	IN WORK





- **ingress_demux:** Splits the 8-channel input stream into two physical memory banks to optimize bandwidth usage.

Storage and Inference Plane

The core memory architecture supports simultaneous read and write operations.

- **ram_bank_0 (The Hot Bank):** Stores Channels 0–3 (12-bit). Acts as the primary source for both AI Inference (continuous high-speed access) and Backend Readout (triggered access).
- **ram_bank_1 (The Cold Bank):** Stores Channels 4–7 (12-bit). Accessed only by the Backend Interface when a full event is triggered.
- **Capacity:** Both banks are circular buffers sized to hold 128 full chunks.

Control Plane (AXI4-Lite)

Managed by a PicoRV32 RISC-V soft-core. It is responsible for task scheduling, interrupt aggregation via `SR_CORE_DONE`, and handling the Epoch Heartbeat.

Data Plane (AXI4 High-Speed)

A 512-bit wide high-bandwidth backbone (`axi4_high_speed_bus`) connecting `ram_bank_0`, uDMA, and the `cnn_core_wrapper` cluster. It ensures a transfer rate of < 30 cycles per chunk.

Compute Cluster

Consists of parallel `cnn_core_wrapper` units (Core 0 to Core N). Each wrapper manages synchronization between the raw data stream and the side-channel timestamp, ensuring that results remain correctly tagged after pipeline latency.

2 Functional Description

2.1 Split-Bank Memory Architecture

To maximize memory bandwidth and eliminate redundant storage, the system employs a Banked Memory Strategy. Instead of separate buffers for storage and inference, the 8-channel signal is split across two parallel RAM banks.

2.1.1 Ingress Signal Splitting (`ingress_demux`)

The demultiplexer separates the assembled 8-channel chunk into two parallel write paths:

- **Path A (Low Channels):** `adc_data[3:0]` are written to `ram_bank_0` at address T_{short} .
- **Path B (High Channels):** `adc_data[7:4]` are written to `ram_bank_1` at address T_{short} .

2.1.2 Banked Memory Roles

`ram_bank_0` (Dual-Use)

- **Inference Task:** The uDMA reads 256 samples from this bank to feed the CNN Cores. Since this bank is only 4 channels wide, the transfer is 50% faster than a full-width read.
- **Trigger Task:** Upon a confirmed event, the uDMA reads this bank to retrieve the lower half of the waveform.

`ram_bank_1` (Storage Only)

- **Trigger Task:** Accessed only when a trigger decision is positive. The uDMA reads this bank to retrieve the upper half of the waveform.



2.1.3 Addressing and Circularity

Both banks share the same write pointer (T_{short}) driven by the `ts_generator`.

- **Write Address:** $Base + (T_{short} \times ChunkSize)$.
- **Overwrite:** The buffer wraps every 128 chunks. The system relies on `SR_WRITE_PTR` to ensure the CPU does not read data that is actively being overwritten.

2.2 Task Orchestration

2.2.1 Dynamic “First-Available” Scheduler

The PicoRV32 firmware implements a low-latency loop to maximize the utilization of parallel cores:

1. **Poll:** The CPU monitors the `ts_generator` to see which Timestamp ID (T) has just been written to memory.
2. **Select:** The CPU reads the `SR_CORE_READY` bitmask to identify the first idle worker (e.g., Core i).
3. **Dispatch:** The CPU calculates the source address in `infer_ram_b` using T and configures the uDMA:
 - `CR_DMA_SRC`: Base Address of `infer_ram_b` + $(T \times \text{Chunk Size})$.
 - `CR_DMA_DEST`: Address of `cnn_core_wrapper[i]`.
 - `CR_DMA_LEN`: Size of one `payload_chunk`.
 - `CR_DMA_CTRL`: Start Bit.
4. **Repeat:** The uDMA handles the transfer on the High-Speed Bus (512-bit width), allowing the CPU to immediately loop back.

2.3 Timing and Wrapper Logic

The system employs a multi-stage timing architecture to reconcile the limited on-chip memory depth with the need for long-term time tracking.

2.3.1 AI Accelerator Wrapper (`cnn_core_wrapper`)

The wrapper acts as an isolation layer, decoupling the high-speed data transport from the deterministic latency of the Neural Network IP.

- **Input Burst Buffering (FIFO256):** Receives data from the 512-bit bus (fill time ≈ 30 cycles) and drains into the IP Core (process time > 1000 cycles).
- **“Just-In-Time” Refill Mechanism:** To eliminate dead time, when `FIFO_LEVEL` < 4 , the Wrapper asserts a hardware `dma_req`. The uDMA immediately bursts the next chunk from `ram_bank_0`, overlapping transfer with processing.
- **Side-Channel Timestamp Synchronization:** The wrapper pushes the Input Timestamp (T_{short}) into a parallel Delay FIFO. When the CNN outputs a valid Score, the Delay FIFO pops the corresponding Timestamp to maintain strict temporal causality.



2.3.2 Two-Tier Timing Engine

Two-Tier Timing is designed to conserve resources by avoiding the tagging of each chunk. Since a large number of chunks may be processed simultaneously within the system, and varying return times occur in systems handling high parallelism and multiple concurrent triggers, it is necessary to number the chunks.

- T_{short} : This is a system-wide timestamp (second-level timestamp) used to assign a relative arrival time to each chunk. Its maximum value is 128. It is immediately assigned to a chunk when ADC data enters the trigger system and remains unchanged throughout the inference and data flow. Only after the backend system receives the chunk is it reused. This numbering is incremented cyclically. The relative sequence number output within the same cycle represents the relative order of chunk arrival. If a chunk triggers, all its channels are output with this timestamp; if it does not trigger, no output occurs.
- T_{epoch} : This is the primary timestamp used to mark the number of T_{short} cycles. Its maximum value is 512. It spans the entire data preprocessing and transmission system. Its sequence number increments by 1 each time a T_{short} cycle is used. Unlike T_{short} , its sequence number always increments by 1 regardless of whether an output was triggered in the current cycle. This timestamp enables monitoring of the timing system's proper operation during output and tracking of data's relative timing. The backend can quickly obtain real-world time on the host machine using simple algorithms and network connectivity.

2.4 Trigger and Re-Ordering Logic

2.4.1 Software Re-Order Buffer

Since the parallel CNN cores operate asynchronously, inference results may complete out of order (e.g., the result for Timestamp $T = 5$ might arrive before $T = 4$). To maintain a causal, time-ordered data stream for the backend, the PicoRV32 implements a **Software Re-Order Buffer**.

- **Ingest:** When a Core finishes (indicated by `SR_CORE_DONE`), the CPU reads the result {Score, Timestamp}.
- **Buffering:** Instead of acting immediately, the CPU stores the Score in a software array: `pending_results[Timestamp] = Score`.
- **Ordering:** The CPU maintains a pointer `next_expected_ts`. It only processes the result at this pointer. If the result for `next_expected_ts` is not yet available, the CPU waits, ensuring strictly ordered processing.

2.4.2 Thresholding and Backend Data Move

Once the result for `next_expected_ts` becomes available, the CPU executes the Trigger Decision Logic:

Threshold Check Compare Score vs. `TRIGGER_THRESHOLD` (configurable).

Positive Trigger (> Threshold)

The system must readout the **Full 8-Channel Event** corresponding to this timestamp.

1. **Transfer 1 (Low Channels):** CPU commands uDMA to move 256 samples from `ram_bank_0` (Address T) to `backend_tx_if`.
2. **Transfer 2 (High Channels):** CPU immediately commands uDMA to move 256 samples from `ram_bank_1` (Address T) to `backend_tx_if`.

Result: The backend receives the complete high-fidelity waveform.

Negative Trigger (< Threshold)

No action is taken; data in RAM is simply left to be overwritten by the circular buffer.

Advance The `next_expected_ts` pointer is incremented (modulo 128), and the loop repeats.



3 Memory Map and Register Description

3.1 Global Control Registers

Table 1: Global Control Registers Map

Offset	Name	Access	Width	Description
0x00	CR_DMA_CTRL	R/W	32	Bit 0: START (Write 1 to dispatch). Bit 1: IRQ_CLR (Write 1 to clear). Bit 2: SOFT_RESET (Flush FIFOs).
0x04	CR_DMA_SRC	R/W	32	uDMA Source Address.
0x08	CR_DMA_DEST	R/W	32	uDMA Destination Address.
0x0C	CR_DMA_LEN	R/W	32	Transfer Length (Bytes).
0x10	SR_CORE_READY	R	32	Global Bitmask: Bit $i = 1$ implies Core i is IDLE.
0x14	SR_CORE_DONE	R/W1C	32	Interrupt Aggregator: Bit $i = 1$ indicates Core i finished. CPU writes 1 to clear.
0x18	SR_WRITE_PTR	R	32	Safety Monitor: Current T_{short} write pointer.
0x2C	CR_EMERGENCY	R/W	32	Fault Config: [7:0] Batch Size (Def 10). [31] Force Active (Manually trigger Emergency Mode).

3.2 Core-Specific Registers

Note: These registers are repeated for each core (Core 0 to Core N). The Base Address for Core i is $\text{Global_Base} + 0x100 + (i * 0x10)$.

Table 2: Core-Specific Registers (Offset from Core Base)

Offset	Name	Access	Width	Description
0x00	CR_CORE_CTRL	R/W	32	Bit 0: START (Dispatch inference). Bit 1: IRQ_CLR (Ack interrupt). Bit 2: SOFT_RESET.
0x04	SR_CORE_RESULT	R	32	[7:0]: Score (Confidence 0-255). [20:8]: Timestamp (T_{short}).
0x08	SR_CORE_STATUS	R	32	Bit 0: BUSY (Inference in progress). Bit 1: DONE (Valid result waiting).

4 Timing and Performance Constraints

- **Throughput Requirement:** The system must process one chunk every 256 ns.
- **Bus Bandwidth:**
 - The AXI4 High-Speed Bus must be configured with a width of **512 bits**.
 - **Transfer Budget:** At 512-bit width, one inference chunk (12,288 bits) should be transferred in < 30 clock cycles.
 - **Refill Latency:** The uDMA arbitration and bus transfer must complete within the processing window of the final 4 samples of the previous chunk to prevent core starvation.
- **Concurrency:** The system supports N parallel cores. The uDMA bandwidth allows saturating up to ≈ 8 cores simultaneously before bus contention becomes a limiting factor.



5 Operational Modes

The system implements a state machine managed by the PicoRV32 Control Plane.

5.1 Idle Mode

- **Status:** System is powered and responsive to AXI4-Lite commands; data processing is halted.
- **Behavior:** `ingress_subsys` ignores data. uDMA is disabled. Interrupts masked.
- **Use Case:** Configuration (setting thresholds, loading weights) and standby.

5.2 Run Mode

- **Status:** Fully active AI-based triggering.
- **Behavior:** uDMA automatically fetches chunks. CPU executes "Re-Order & Threshold" loop. Only chunks with `Score > Threshold` are sent to backend.
- **Use Case:** Standard data-taking for physics experiments.

5.3 Passthrough Mode

- **Status:** AI Inference is bypassed.
- **Behavior:** CPU forces a trigger event periodically (e.g., every k -th chunk). Unbiased raw waveforms are sent.

6 System Initialization and Error Recovery

6.1 Cold Boot Sequence

Upon power-up or Hardware Reset (`rst_n`), the firmware must execute:

1. **Global Masking:** Write `0xFFFFFFFF` to `SR_CORE_DONE` to clear spurious interrupts. Disable uDMA.
2. **Core Sanity Check:** Iterate through Cores `0..N`, write `SOFT_RESET`, and verify `SR_CORE_STATUS` reads `READY`.
3. **Memory Initialization:** Zero-out the `pending_results` buffer. Reset software pointers.
4. **Transition:** Enter **Idle Mode**.

6.2 Run Start Procedure

To transition from Idle to Run Mode:

1. **Configuration:** Load Trigger Threshold and reset Performance Counters.
2. **Ingest Enable:** Enable the `ts_generator`.
3. **Sync Wait:** CPU waits for the first `irq_epoch` interrupt to align chunk processing with Timestamp 0.
4. **Loop Start:** Enable Global Interrupts and enter the main Scheduler Loop.



A Appendix

A.1 Term and Parameter Definitions

The following terms and configurable parameters are used throughout the specification to define the system architecture and timing constraints. These parameters are listed in *italics*.

Table 3: System Terms and Definitions

Term	Definition
ADC	Analog-to-Digital Converter. The front-end hardware component responsible for digitizing analog waveform signals into 12-bit discrete samples.
Chunk	Payload Chunk. The fundamental unit of data processing. One chunk consists of 256 contiguous time samples across all 8 channels. The Size is 12 bits per sample per channel, and the ADC Digitizer generates 1 sample per 4 channels per nanoseconds.
Emergency Mode	A fail-safe operational state triggered when processing lag drops below the safety headroom ($\text{Gap} < 10$). The system automatically triggers (force-accepts) chunks to clear the backlog.
Epoch (T_{epoch})	A coarse-grained 9-bit counter used to track time over longer durations. Increments every time T_{short} wraps (every 128 chunks). Used for backend time reconstruction.
Heartbeat	A synchronization packet (Type B) containing the current T_{epoch} , transmitted every 128 chunks to keep the Backend DAQ synchronized with the FPGA clock.
Lag	The distance (in chunks) between the current Write Pointer (Ingress) and the Read Pointer (Inference). Formula: $(SR_WRITE_PTR - next_expected_ts) \% 128$.
PicoRV32	The soft-core RISC-V processor acting as the Control Plane. It manages task scheduling, interrupts, and register configuration via AXI4-Lite.
Score	An 8-bit confidence value (0–255) generated by the CNN Accelerator Core, indicating the likelihood that a chunk contains a physics event.
Timestamp (T_{short})	A fine-grained 7-bit cyclic counter (0–127) tagging every chunk. Serves as the physical memory address index for the circular buffers.
Trigger Threshold	A configurable parameter. If a chunk's <i>Score</i> exceeds this value, the system triggers a readout of the full high-fidelity waveform.
uDMA	Micro Direct Memory Access. A hardware engine moving bulk data between memory banks and peripherals (CNN Cores, UART) via the 512-bit High-Speed Bus without CPU intervention.